NutriAI

Assessment Model — Input, Output & Wellness Score

How diary entries become model features, how the Random Forest returns deficiency probabilities, and how the wellness score is calculated.

Model: Random Forest (MultiOutputClassifier) · Trained on NHANES 2021–2023 · Artifacts: deficiency_model.joblib, preprocess.joblib, meta.json

1. Overview

The Assessment page does not send individual foods or meal names to the machine-learning model. It sends one numeric row per user: age, gender, and average daily nutrient intake from the food diary.

The model was trained as a multi-label classifier (10 binary deficiency labels). At inference the app does not use the 0/1 class. It uses predict_proba — the model's probability that each nutrient is deficient.

```
diary_entries  →  daily nutrient totals  →  average per day
combine with age + gender  →  13 features
Random Forest predict_proba  →  10 deficiency probabilities (0–1)
optional lab boost  →  Assessment UI + wellness score
```

2. End-to-end data flow

Step 1 — User logs food

When a meal is added, nutrients are computed from food_items (always stored per 100 g) and scaled by the logged portion:

```
nutrient_logged = (nutrient_per_100g / 100) × portion_grams
```

The result is saved on that diary_entries row. The model is not called at log time.

Step 2 — User clicks Run assessment

Frontend calls POST /api/assess. Backend requires at least 3 distinct diary days (default window: last 7 days) and a completed health profile.

Step 3 — Aggregate diary into daily averages

get_diary_summary() reads diary_entries for the current user, sums nutrients per calendar day, then averages across distinct logged days.

```
Day 1 iron + Day 2 iron + … + Day N iron
average_iron = total / N   (same for all 11 nutrient fields)
```

Step 4 — Build the 13-feature input

build_feature_dict() merges health profile (gender, age) with the diary average. This dictionary is the only input the model receives.

Step 5 — Preprocess + Random Forest

Features are ordered exactly as in meta.json, imputed and scaled with preprocess.joblib (same pipeline as training), then passed to model.predict_proba().

Step 6 — Optional lab boost

Lab values are not model inputs. After prediction, low labs can raise selected risk scores (iron, vitamin D, B12, calcium).

Step 7 — Save and display

Results are stored in assessments (features_json, risks_json, overall_score) and shown on the Assessment page. Recommended foods are a separate rule-based query on food_items, not an ML output.

3. Model input (13 features)

The model never sees food names, meal types, or individual diary rows. Input is always this fixed feature list from meta.json:

#	Feature	Source	Meaning
1	gender	health_profiles	1 = male, 2 = female
2	age_years	health_profiles	Age in years
3	calories_kcal	diary average	Avg daily energy
4	protein_g	diary average	Avg daily protein
5	iron_mg	diary average	Avg daily iron
6	calcium_mg	diary average	Avg daily calcium
7	vitamin_d_mcg	diary average	Avg daily vitamin D
8	vitamin_b12_mcg	diary average	Avg daily B12
9	folate_mcg	diary average	Avg daily folate
10	vitamin_a_mcg	diary average	Avg daily vitamin A
11	vitamin_c_mg	diary average	Avg daily vitamin C
12	magnesium_mg	diary average	Avg daily magnesium
13	zinc_mg	diary average	Avg daily zinc

Worked example (real assessment)

User haydicaf@fosil.pro, 5 diary days. Actual feature vector sent to the model:

Feature	Value
gender	2 (female)
age_years	25
calories_kcal	945.784
protein_g	51.411
iron_mg	12.734

calcium_mg	380.297
vitamin_d_mcg	1.99
vitamin_b12_mcg	0.36
folate_mcg	348.136
vitamin_a_mcg	268.942
vitamin_c_mg	124.998
magnesium_mg	298.108
zinc_mg	7.666

As a numeric row (exact order the model expects):

```
[2, 25, 945.784, 51.411, 12.734, 380.297, 1.99, 0.36,
 348.136, 268.942, 124.998, 298.108, 7.666]
```

4. Labels — these are the outputs

Labels are not extra inputs. They are the 10 targets the model was trained to predict. Each label is binary:

```
0 = not deficient
1 = deficient
```

Label names (from meta.json):

Label (output name)	How the training label was created
label_iron	Lab: low Hb (M<13 / F<12) OR ferritin<30; else iron intake < 8 mg
label_vitamin_d	Lab: serum Vit D < 50; else intake < 15 mcg
label_vitamin_b12	Intake B12 < 2.4 mcg
label_calcium	Intake calcium < 1000 mg
label_folate	Intake folate < 400 mcg
label_vitamin_a	Intake Vit A < 700 mcg
label_vitamin_c	Intake Vit C < 75 mg
label_magnesium	Intake magnesium < 320 mg
label_zinc	Intake zinc < 8 mg
label_protein	Intake protein < 50 g

During training, NHANES labs/intake were converted into these 0/1 labels. The model learned to predict them from the 13 features. Labs are not used as model features.

5. Two kinds of output: label vs probability

For the same input, scikit-learn can return either the class or the probability. The app only calls predict_proba. It never calls predict().

Method	What it returns	Meaning	Used in app?
model.predict(X)	0 or 1	Predicted label (deficient or not)	No
model.predict_proba(X)	0.00 to 1.00	P(label = 1) = P(deficient)	Yes

Relationship between them:

```
if probability >= 0.5  →  label = 1 (deficient)
if probability <  0.5  →  label = 0 (not deficient)
```

Example: calcium probability 0.988 implies label_calcium = 1. The UI shows 99%, not the integer 1, because the probability is more informative.

predict_proba is given by the model

We do not calculate the 99% ourselves with an RDA formula. We call the trained Random Forest's built-in predict_proba(). Internally, many trees vote 0 or 1; the share that vote "deficient" is the probability.

```
100 trees for calcium
  99 vote deficient (1)
   1 vote not deficient (0)
predict_proba = 99/100 = 0.988  →  UI shows 99%
```

What our code does after the model returns the number:

Step	Who	What
Build 13 features	Our code	Diary average + age/gender
Impute + scale	preprocess.joblib	Same pipeline as training
predict_proba()	Random Forest model	Returns 10 probabilities
Read class 1	Our code	probs[0][1] = P(deficient)
Show % on UI	Our code	0.988 × 100 = 99%
Lab boost	Our code	May raise iron / Vit D / B12 / calcium

6. Model output (10 probabilities + implied labels)

Raw model output for the same worked example, before lab boost. Each row is one label.

Label	predict() 0/1	predict_proba P(deficient)	Shown on UI
label_calcium	1	0.988	99% HIGH
label_zinc	1	0.985	99% HIGH
label_vitamin_b12	1	0.980	98% HIGH
label_folate	1	0.939	94% HIGH
label_magnesium	1	0.932	93% HIGH
label_vitamin_a	1	0.929	93% HIGH
label_iron	0	0.488	49% (then lab-boosted)
label_vitamin_d	0	0.432	43% (then lab-boosted)
label_protein	0	0.118	12% LOW
label_vitamin_c	0	0.039	4% LOW

The percentage on the Assessment page is this probability × 100. HIGH / MODERATE / LOW is only a UI band, not a separate model output:

Probability	UI level
< 0.35 (< 35%)	LOW
0.35 to 0.65 (35–65%)	MODERATE

> 0.65 (> 65%)	HIGH

The green tag under each nutrient (for example “Low calcium intake”) is not from the model. It is a simple check of diary average vs an RDA-style threshold.

7. Lab boost (after the model)

Optional lab_results can raise a risk after predict_proba. They never replace the model and are never sent as features.

Lab field	Cutoff	Boost applied
hemoglobin_g_dl	Female < 12, male < 13	iron = max(model, 0.75)
ferritin	< 30	iron = max(model, 0.70)
serum_vitamin_d	< 50	vitamin_d = max(model, 0.65)
vitamin_b12	< 200	vitamin_b12 = max(model, 0.60)
calcium	< 8.5	calcium = max(model, 0.55)

Worked example labs for haydicaf@fosil.pro:

Nutrient	Model P	After lab boost	UI
Iron	0.488	0.75 (Hb 10.5 + ferritin 22)	75% HIGH
Vitamin D	0.432	0.65 (serum Vit D 18)	65% MODERATE
Other 8 nutrients	unchanged	unchanged	as model

8. Wellness score calculation

The wellness score is not a separate model output. It is computed from the 10 (lab-boosted) probabilities:

```
avg_risk = (P1 + P2 + … + P10) / 10
overall_score = (1 − avg_risk) × 100
```

In code (app/routers/assess.py):

```
avg_risk = sum(r["probability"] for r in risk_items) / len(risk_items)
overall_score = round((1 - avg_risk) * 100, 1)
```

Worked example matching the Assessment page (26.9%):

Nutrient	Probability used
Calcium	0.988
Zinc	0.985
Vitamin B12	0.980
Folate	0.939
Magnesium	0.932
Vitamin A	0.929
Iron (boosted)	0.750
Vitamin D (boosted)	0.650
Protein	0.118

Vitamin C	0.039

```
sum = 7.310
avg_risk = 7.310 / 10 = 0.731
wellness = (1 − 0.731) × 100 = 26.9%
```

Interpretation: high average deficiency risk → low wellness score. 26.9% means most of the 10 nutrients are high-risk for this user.

9. What is and is not from the model

Item on Assessment page	From the model?
99% / 98% / 12% risk percentages	Yes — predict_proba × 100 (except lab-boosted ones)
Implied 0/1 labels	Yes — can be derived from probability ≥ 0.5
HIGH / MODERATE / LOW badge	No — UI threshold on the probability
“Low calcium intake” tag	No — diary vs RDA check
Wellness score 26.9%	No — formula on the 10 probabilities
Recommended foods list	No — query food_items by nutrient + diet flags
“5 diary days · min 3 required”	No — diary_entries count

10. One-line summary

```
INPUT  (13):  gender, age, avg daily calories + 10 nutrients
OUTPUT (10):  label_iron … label_protein
              each as P(deficient) from model.predict_proba()
              0/1 label = 1 if P ≥ 0.5
UI %:         P × 100   (e.g. 0.988 → 99%)
WELLNESS:     (1 − average of 10 probabilities) × 100
```

Educational / internship demonstration only. Not a medical device and not clinical advice.